

# Juke in a Nutshell

---

## *A Desktop Quick Reference*

### **The Juke Project**

*Early Release — covers Juke 0.0.1*

*Beijing · Boston · Farnham · Sebastopol · Tokyo*



*The animal on the cover of Juke in a Nutshell is a superb lyrebird (Menura novaehollandiae), one of nature's great mimics — it records the sounds of its world and replays them with uncanny fidelity, much as Juke records and replays your upstream interactions.*

# Table of Contents

## Preface

## Part I — Understanding Juke

1. Introduction to Juke
2. Getting Started
3. How Juke Works
4. The Annotation Surface
5. Operating Modes
6. Sessions and Per-Track Replay
7. The Recording Format
8. Remix: Runtime Fault Injection
9. Functional Test Coverage
10. Session Reports

## Part II — Reference

11. The `/service/*` HTTP Contract
12. Configuration Reference
13. The Storage SPI

## Part III — Appendices

- Appendix A. Sample Applications
- Appendix B. Troubleshooting
- Appendix C. Glossary

## Colophon

---

# Preface

---

Test code spends an extraordinary amount of its life pretending. A unit test pretends an inventory service returned three widgets; an integration test pretends a payment gateway timed out; a UI test pretends the weather API is having a good day. Every one of those pretences is hand-written, drifts away from reality the moment the real system changes, and has to be maintained forever.

*Juke* replaces the pretending with *recording*. It captures what real upstream systems actually did, stores it in a portable archive, and replays it deterministically — so your tests run against real data with no mock code to write or own.

This book is a quick reference for developers and test engineers who use, or are evaluating, Juke. It covers the open-source **Community** distribution in full and points to where the **Enterprise** modules extend it.

## Who This Book Is For

You should be comfortable with Java and Spring Boot. You do not need prior experience with mocking frameworks, service virtualization, or recording tools — Juke's model is explained from first principles in Chapter 3. If you write Playwright, Cypress, or Selenium tests against a Spring Boot backend, Part I will be immediately useful; if you operate Juke in CI, Part II is your desk reference.

## How This Book Is Organized

The book is deliberately layered. **Part I** builds understanding from the top down: each chapter opens with the general idea and why it matters, then descends into nuance. Read it front to back the first time.

- **Chapter 1** makes the case for Juke and surveys the distribution.
- **Chapter 2** gets a recording replaying in five minutes.
- **Chapter 3** explains the mental model — the interception seam, the modes, the recording.
- **Chapters 4–8** are the working core: annotations, modes, sessions, the recording format, and runtime fault injection.
- **Chapters 9–10** cover the newer reporting surface — functional coverage and session reports.

**Part II** is dense reference: the complete HTTP contract, every configuration key, and the storage SPI. Reach for it once the concepts in Part I are familiar.

**Part III** collects the sample applications, a troubleshooting catalog, and a glossary.

## Conventions Used in This Book

The following typographic conventions are used:

*Italic*

New terms, filenames, and URLs.

`Constant width`

Code, annotations, configuration keys, command-line input, and the names of classes and methods.

`Constant width italic`

Text that should be replaced with a user-supplied value.

**Note** — A note flags a useful aside or a point of clarification.

**Tip** — A tip is a recommended practice or a shortcut.

**Warning** — A warning calls out a sharp edge that can cost you time.

## Using Code Examples

Every command, annotation, and configuration snippet in this book is copy-pasteable. Examples target Juke 0.0.1 on Java 25 and Spring Boot 3.5. Where a command differs between shells, both forms are given.

---

# Part I — Understanding Juke

---

---

# Chapter 1. Introduction to Juke

---

This chapter answers two questions: *what is Juke*, and *why would you choose it over the testing tools you already have*. The rest of the book assumes the answers.

## 1.1 What Juke Is

Juke is a **record-and-replay framework for Spring Boot services**. It sits between your service-layer beans and the upstream collaborators they call — other services, databases, REST endpoints, message brokers, anything reachable through a Spring-injected interface or `*Template`.

In **record** mode it captures every upstream interaction as a JSON document inside a portable ZIP archive. In **replay** mode it serves those documents back deterministically, so no upstream call ever leaves the host process. The result is a deterministic test fixture made of *real* data: UI tests, integration tests, and CI pipelines all see byte-identical responses run after run, regardless of what the underlying upstream system is doing.

Juke is two ideas working together:

- A small set of **annotations** — `@Juke`, `@JukeController`, `@JukeIgnorable` — that mark which beans to intercept.
- A **mode flag** — `record` / `replay` / `ignore` — that decides what the interceptor does.

Everything else in this book is detail hanging off those two ideas.

## 1.2 Why Juke

Unit-testing frameworks (Mockito, EasyMock) and stub/behaviour tools (WireMock, MockServer, Spring Cloud Contract) all make you *author the fake yourself*. You write `when(...).thenReturn(...)`, you hand-craft stub JSON, or you maintain a separate contract file. That fake is only ever a guess about what the real upstream does; it is boilerplate to write and own; and it drifts silently out of sync the moment the upstream changes.

Juke removes the fake entirely. Four properties follow:

**Nothing to author or maintain.** One annotation — `@Juke` — and the test data is captured from the real upstream. No `when/thenReturn`, no stub files, no contract authoring, nothing to keep in sync.

**Real data, not assumptions.** A recording is the upstream's genuine response — its real edge cases, nulls, and encodings included. A hand-written mock returns only what its author imagined; a Juke recording returns what the system actually produced.

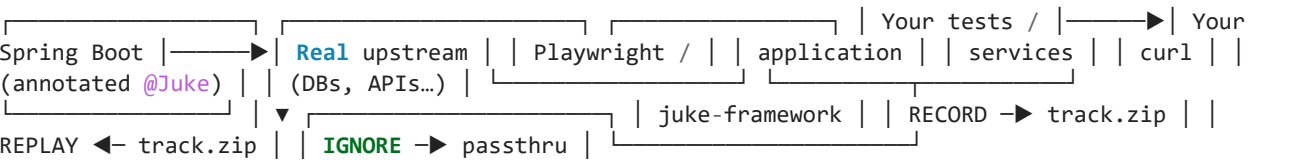
**It exercises your real code.** A Mockito mock replaces the whole bean, so your serialization, retry logic, circuit breakers, and the Spring proxy chain never run under test. Juke replaces *only* the I/O at the

upstream edge — every line of your own code on the path still executes. That is what makes a Juke replay a genuine *behavioural* test rather than a unit-isolated one.

**Deterministic and parallel-safe.** Replay is byte-identical on every run, which removes the root cause of flaky tests; and cookie-scoped sessions let many test workers each replay a different recording against a single JVM with no shared state.

**Note** — Put plainly: Mockito tests your code against *your assumptions*, WireMock tests it against *a stub you maintain*, and Juke tests it against *what the real system actually did*.

### 1.3 Where Juke Fits



Juke does not replace Playwright, JUnit, or your CI system — it sits beneath them, supplying the server-side data layer they have always had to fake.

### 1.4 The Juke Distribution

The Community distribution is four Apache 2.0 jars, plus an optional fifth:

| Module                  | Role                                                                                  |
|-------------------------|---------------------------------------------------------------------------------------|
| juke-framework          | The engine — annotations, proxies, the storage SPI, the session registry.             |
| juke-remix-rest-service | The <code>/service/*</code> HTTP control surface — record, replay, sessions, reports. |
| juke-plugin-api         | DTOs for the plugin contract.                                                         |
| juke-plugin-sdk         | A Spring Boot starter for writing plugins.                                            |
| juke-coverage           | <i>Optional</i> . Functional code-coverage capture (Chapter 9).                       |

The Enterprise modules — `juke-scenario-service` (JPA persistence, database-backed storage), `juke-admin-server`, and the `juke-admin-ui` SPA — are strict additions; nothing in Community changes when you bring them in.

## 1.5 How Juke Compares

| Capability                          | Juke | Mockito | WireMock | Spring Cloud Contract |
|-------------------------------------|------|---------|----------|-----------------------|
| Zero hand-written fakes             | ✓    | ✗       | ✗        | ✗                     |
| Records from live traffic           | ✓    | ✗       | ⚠ manual | ✗                     |
| Exercises the real code path        | ✓    | ✗       | ✓        | ✓                     |
| Deterministic sequenced replay      | ✓    | ⚠       | ⚠        | ✗                     |
| Concurrent session isolation        | ✓    | N/A     | ✗        | ✗                     |
| Runtime fault injection, no restart | ✓    | ⚠       | ✓        | ✗                     |
| Portable archive of recordings      | ✓    | ✗       | ✓        | ✓                     |



# Chapter 2. Getting Started

---

This chapter takes a Spring Boot application from "no Juke" to "replaying a recording" in about five minutes. The concepts behind each step are deferred to Chapter 3 — here the goal is a working result.

## 2.1 Prerequisites

- Java 25
- Maven 3.9 or newer
- An existing Spring Boot 3.x application, or one of the bundled samples under `juke-samples/` (see Appendix A)

## 2.2 Add the Dependencies

```
<dependency> <groupId>org.juke.harness</groupId> <artifactId>juke-framework</artifactId>  
<version>0.0.1-SNAPSHOT</version> </dependency> <dependency> <groupId>org.juke.harness</groupId>  
<artifactId>juke-remix-rest-service</artifactId> <version>0.0.1-SNAPSHOT</version> </dependency>
```

`juke-framework` carries the engine; `juke-remix-rest-service` adds the `/service/*` control surface. Open the component scan so both are found:

```
@SpringBootApplication @ComponentScan(basePackages = {"org.juke.framework", "org.juke.remix",  
"com.yourcompany"}) public class YourApplication { ... }
```

## 2.3 Turn Juke On

Juke is **off by default**. A single YAML key opts an environment in:

```
juke: enabled: true # master switch path: /tmp/juke # directory holding recording ZIPs
```

**Warning** — When `juke.enabled` is absent or `false`, the framework wires *nothing*: no proxies, no session beans, no `/service/*` controllers. Keep the key out of `application-prod.yml` so production stays Juke-free.

## 2.4 Annotate One Collaborator

```
@Service public class CheckoutService { @Juke private InventoryClient inventory; // ◀ this  
dependency is intercepted public Order place(String sku, int qty) { return inventory.reserve(sku,  
qty); } }
```

## 2.5 Record, Then Replay

Boot the app, then drive the lifecycle over REST — no restarts:

```
# Record curl "http://localhost:8080/service/record/start?track=morning-flow" curl
"http://localhost:8080/checkout?sku=A1" curl "http://localhost:8080/service/record/end" -o
morning-flow.zip # Replay, scoped to one test session curl -c cookies.txt
"http://localhost:8080/service/session/start?track=morning-flow" curl -b cookies.txt
"http://localhost:8080/checkout?sku=anything" curl -b cookies.txt
"http://localhost:8080/service/session/stop"
```

The replayed call returns the recorded response regardless of the `sku` you send. That is the deterministic guarantee — and the rest of Part I explains how it works.

---

# Chapter 3. How Juke Works

---

Chapter 2 produced a result; this chapter supplies the model behind it. Once you hold the model, every endpoint, mode, and file in the later chapters has an obvious place to hang.

## 3.1 The Interception Seam

Juke does its work at a single point: the **@Juke-annotated injection point**. When Spring finishes wiring a bean, Juke's **BeanPostProcessor** replaces the annotated field's value with a *proxy* of the same type. The proxy implements the field's interface (or subclasses its class) and intercepts every method call.

Nothing else in the application is aware of this. The controller, the service logic, the serialization layer — all of it runs unchanged and unmodified. The *only* thing that changes is what happens when execution crosses the seam.

## 3.2 Record, Replay, Ignore

At the seam, the proxy consults the current **mode** and acts:

| Mode   | At the seam                                                                                                      |
|--------|------------------------------------------------------------------------------------------------------------------|
| record | Call the real upstream; capture the response, arguments, and type into the active ZIP; return the real response. |
| replay | Do <i>not</i> call the upstream; read the next recorded response from the ZIP, deserialize it, and return it.    |
| ignore | Call the real upstream; capture nothing. Pure pass-through.                                                      |

Modes are covered fully in Chapter 5. The key idea: the application code is identical in all three; only the proxy's behaviour at the seam differs.

## 3.3 Anatomy of an Interception

Each intercepted call produces a recording **identifier**:

<TypeName>.<methodName>.<sequence>

- *TypeName* — the interface simple name for **@Juke** on a field; the concrete class simple name for **@Juke** on a class.

- *methodName* — the Java method name; overloads are disambiguated by a parameter-signature hash.
- *sequence* — a 1-based per-(type, method) counter.

Three calls to `IGreetingsService.greeting(...)` in record mode produce `IGreetingsService.greeting.1`, `.2`, and `.3`. In replay mode the counter resets at session start, so the same three calls return the same three responses in the same order.

**Note** — Replay is *sequenced*, not *keyed by argument*. The fourth call to a method that was recorded three times has nothing to return — see "JukeReplayNotFoundException" in Appendix B.

### 3.4 The Whole Picture

@Juke field mode = record mode = replay ————— proxy.call() →  
 real upstream → ZIP proxy.call() → ZIP | | ⊥ response returned ⊥ recorded response to caller  
 returned to caller

Record once against the real world; replay forever against the archive. The remaining chapters add precision: which beans (Chapter 4), which mode and how it is chosen (Chapter 5), how concurrent tests stay isolated (Chapter 6), what the archive contains (Chapter 7).

---

# Chapter 4. The Annotation Surface

---

Three annotations cover virtually every Spring injection pattern, and a fourth opts individual members out. This chapter is the working reference for all four.

## 4.1 @Juke

The workhorse. Its targets are `TYPE`, `FIELD`, `METHOD`, and `PARAMETER`; retention is `RUNTIME`.

**On a field whose type is an interface** — Juke installs a JDK dynamic proxy:

```
@Service public class OrderService { @Juke private InventoryClient inventory; @Juke private PricingClient pricing; }
```

**On a field whose type is a concrete class** (e.g. a Spring `RestTemplate`) — Juke installs a CGLIB subclass that delegates to the real, fully-configured bean, so the wrapper is assignable back to the field. Two optional attributes apply to the concrete case:

```
@Autowired @Juke(name = "shipping", excludeMethods = {"setMessageConverters"}) private RestTemplate restTemplate;
```

`name` overrides the recording-identity prefix (defaulting to the type's simple name) so two beans of the same type don't collide; `excludeMethods` skips config/builder methods you don't want recorded. A `final` concrete class can't be subclassed — type the field as the interface it implements instead. Per-session cookie replay currently applies to interface-typed fields only.

**On a class** — Juke installs a CGLIB subclass proxy, and *every* public method of the class and of every interface it implements is intercepted:

```
@Juke @Service public class FulfillmentService implements Shippable, Trackable { ... }
```

Recordings for a class-level `@Juke` are keyed by the **concrete class name** (`FulfillmentService.ship.1`), not the interface name. `Object.equals`, `hashCode`, and `toString` are never intercepted.

**On a method or parameter** — Juke wraps the parameter before the method body runs, or wraps the return value after it returns.

The optional `value()` attribute pins this annotation to a mode regardless of the global setting:

```
@Juke("ignore") private DiagnosticsClient diagnostics; // never recorded – always pass-through
```

Accepted values: `juke` (default — follow the global mode), `record`, `replay`, `ignore`, `none`, `disable`. The `autoWrap()` attribute (default `true`) disables wrapping when set `false`.

## 4.2 @JukeController

`@JukeController` marks a Spring `@RestController` for controller-level AOP advice: request/response logging in MDC tags, optional finding capture, and cookie-bound session resolution. Most applications

never apply it directly — it is wiring for the bundle-backed session flow (Chapter 6).

### 4.3 @JukeIgnorable

When `@Juke` is on a class, every public method is wrapped. `@JukeIgnorable` opts one member out:

```
@Juke @Service public class OrderService { public Order place(String sku) { ... } // recorded
@JukeIgnorable public void warmCache() { ... } // never recorded }
```

It also drives field-level comparison rules. Its `IgnoreStrategy` has two values:

| Strategy                      | Behaviour                                                                               |
|-------------------------------|-----------------------------------------------------------------------------------------|
| <code>ALWAYS</code> (default) | Skip this field in every comparison — right for generated IDs and UUIDs.                |
| <code>NOT_NULL</code>         | Skip the value diff, but still flag a null-vs-non-null mismatch — right for timestamps. |

# Chapter 5. Operating Modes

A mode decides what the proxy does at the seam. This chapter covers the five modes, the three ways to set the global mode, and the precedence rule that lets per-session replay coexist with a global mode.

## 5.1 The Five Modes

| Mode                 | Behaviour                                                                                           |
|----------------------|-----------------------------------------------------------------------------------------------------|
| <code>juke</code>    | Default. Follows whatever mode is active elsewhere.                                                 |
| <code>record</code>  | Calls hit the real upstream; responses, arguments, and type metadata are written to the active ZIP. |
| <code>replay</code>  | Calls are intercepted before the upstream; recorded JSON is read back and returned.                 |
| <code>ignore</code>  | Pass-through. Calls hit the real upstream; nothing is recorded.                                     |
| <code>disable</code> | Master kill-switch. No interception at all — equivalent to removing every <code>@Juke</code> .      |

## 5.2 Setting the Global Mode

With `juke.enabled: true`, the global mode can be set three ways, later winning:

1. **REST at runtime (recommended).** `GET /service/record/start?track=name` flips to record; `GET /service/replay/start?track=name` flips to replay. No restart.
2. **YAML.** `juke.mode: record` in `application.yml`.
3. **System property.** `-Djuke=record` (a legacy alias; YAML is preferred).

## 5.3 Per-Session Versus Global

A Juke proxy checks for an active **session cookie before** it consults the global mode. The two mechanisms are layered, not exclusive:

| Caller | Behaviour |
|--------|-----------|
|--------|-----------|

|                                                  |                                                                       |
|--------------------------------------------------|-----------------------------------------------------------------------|
| Carries a valid <code>JUKE_SESSION</code> cookie | Replay against the session's track. <i>Global mode is irrelevant.</i> |
| No cookie, global mode <code>ignore/none</code>  | Pass-through.                                                         |
| No cookie, global mode <code>record</code>       | Capture into the active track.                                        |
| No cookie, global mode <code>replay</code>       | Replay against the globally-pinned track.                             |
| Any caller, global mode <code>disable</code>     | Pass-through. The kill-switch wins.                                   |

This is why a test session can replay locally while production traffic on the same JVM stays pass-through.

## 5.4 Argument Validation

On replay, Juke compares each call's incoming arguments against the recorded arguments. `juke.args-validation` controls the response:

| Value                       | On mismatch                                                             |
|-----------------------------|-------------------------------------------------------------------------|
| <code>off</code>            | Nothing.                                                                |
| <code>warn (default)</code> | Logs <code>INPUT MISMATCH [...]</code> ; the test continues.            |
| <code>strict</code>         | Throws <code>JukeReplayMismatchException</code> (surfaced as HTTP 500). |

Use `strict` in gate jobs to turn silent input drift into a loud failure.

---



# Chapter 6. Sessions and Per-Track Replay

---

A process-wide mode flip serves one consumer at a time. The session API gives every test worker or browser tab its own track, in parallel, against one JVM.

## 6.1 Why Sessions Exist

Three Playwright workers running in parallel each need a different track — `happy-path`, `cancel-flow`, `retry-after-timeout`. Without sessions you would have to `replay/start` between every worker's request, serializing the suite. With sessions, each worker calls `/service/session/start?track=...` once and receives a `JUKE_SESSION` cookie; the framework's cookie filter resolves that cookie to the right track on every subsequent request.

## 6.2 The Session Lifecycle

```
curl -c jc.txt "http://localhost:8080/service/session/start?track=happy-path" curl -b jc.txt  
"http://localhost:8080/checkout?sku=A1" # replays happy-path curl -b jc.txt  
"http://localhost:8080/service/session/stop"
```

`session/start` accepts an optional `description` query parameter that is stored with the session and surfaces in its report (Chapter 10). Each browser context maps 1:1 to a Juke session — in Playwright, set the `JUKE_SESSION` cookie on the context before navigating.

## 6.3 Status Visibility

`GET /service/sessions` returns one row per active session, including `lastCall` (the recording entry most recently served, rendered `<SimpleType>.<method>.<sequence>`) and `percentComplete`, a coarse 0–100 progress signal computed across every entry in the recording. It is built for live status grids.

When a session is stopped it moves into a per-track **history**, which the report endpoint of Chapter 10 reads.

---

# Chapter 7. The Recording Format

---

A Juke recording is an ordinary ZIP archive of JSON. There is no proprietary format and no schema service — you can open one in any unzip tool.

## 7.1 The ZIP Layout

```
happy-path.zip |— IGreetingsService.greeting.1.json ◀ response body |—  
IGreetingsService.greeting.1.args.json ◀ input arguments (sidecar) |—  
IGreetingsService.greeting.1.type.json ◀ runtime return type (sidecar) |—  
IGreetingsService.greeting.2.json | ... |— juke.json ◀ class-metadata catalog |—  
juke-mappings.json ◀ short-name → FQN lookup |— juke-metadata.json ◀ track label + recordedAt
```

## 7.2 Three Files per Call

- **\*.json** — the response, as Jackson serialized it. Replay returns this as the call's return value.
- **\*.args.json** — the input arguments captured at record time. Replay compares incoming arguments against this sidecar (see §5.4).
- **\*.type.json** — the runtime concrete return type, written when it differs from the declared type. Replay uses it to deserialize correctly.

## 7.3 Track Metadata

**juke-metadata.json** records the track's human-readable **label** (supplied via `record/start?...&label=...`) and the timestamp it was recorded. The session report (Chapter 10) reads it so a report can be titled meaningfully.

## 7.4 Why It Is Portable

Plain-text JSON, kilobytes per call, in a single ZIP. Commit recordings to your repository, attach them to bug reports, or share them between teams — no external dependency travels with the file.

---

# Chapter 8. Remix: Runtime Fault Injection

---

Replay reproduces what the upstream *did*. Remix makes the upstream *misbehave* — on demand, on one specific call, against a recording you already trust.

## 8.1 Why Remix Exists

The hardest paths to test are the ones where an upstream is wrong in a specific, controlled way: a service slow exactly once, an exception thrown on exactly the second call. In unit tests these are trivial but unreal — the mock bypasses your retry and circuit-breaker code. In integration and Playwright tests they are nearly impossible without heavyweight chaos tooling. The result is suites that only ever exercise the happy path.

Remix injects precisely the failure you want, on precisely the call you want. The test stays a clean black-box scenario; the upstream misbehaves once, then recovers.

## 8.2 Inject an Exception

```
curl "http://localhost:8080/service/remix/exceptionSchedule?\
classAndMethodSequence=IGreetingsService.greeting.1&\
exception=IOException&exceptionMessage=Simulated+failure"
```

The next replay of that entry throws; the call after returns the recorded payload normally. Schedules are one-shot unless re-armed. This exercises retry catch-blocks, error-banner UX, and logging completeness.

## 8.3 Inject a Delay

```
curl "http://localhost:8080/service/remix/delaySchedule?\
classAndMethodSequence=OrderService.place.3&waitTimeInMS=10000"
```

A delay longer than a configured client timeout drives the timeout branch — code that is otherwise almost never executed. It also exercises loading-state UI and user-perceived-latency handling.

## 8.4 A Test Pattern

To validate "the UI recovers from a single transient failure": load a happy-path recording, schedule a one-shot exception on `Service.call.1`, drive the UI action, and assert the user sees a brief "retrying..." indicator followed by the recorded success. No mocks, no service virtualization, no real network failure — and your production retry code actually runs.

---

# Chapter 9. Functional Test Coverage

---

Coverage tools answer "did my *unit tests* touch this line?" Juke can answer a more valuable question: "do my *real user-journey tests* exercise this code?" This chapter covers the optional `juke-coverage` module.

## 9.1 Use-Case Coverage Versus Unit Coverage

Unit-test coverage is measured against unit tests, which mock away most collaborators. **Functional** (use-case) coverage is measured while Playwright drives real flows through a Juke-replaying server. The *gap* between the two numbers is diagnostic: it reveals undertested workflows and dead code that no real journey reaches.

## 9.2 Server Coverage

`juke-coverage` reads the JaCoCo agent **in-process**. Start the server with the agent attached:

```
-javaagent:jacoco-agent.jar=output=none
```

Then `GET /service/coverage/server` pulls the live execution data, analyzes the application-under-test's classes, renders a drill-down HTML report, and returns a summary:

```
{ "available": true, "passed": true, "tool": "JaCoCo", "instruction": 84.2, "branch": 71.0,
  "line": 86.5, "analyzedClasses": 4, "excludedSeams": ["com.example.IGreetingsService ->
  com.example.GreetingServiceImpl"], "reportUrl": "/coverage/server/index.html" }
```

Because the agent accumulates over the JVM's lifetime, every replay session adds to the same figure — multi-run aggregation is automatic.

## 9.3 Excluding @Juke-Mocked Code

In replay mode, a `@Juke`-mocked implementation never executes — counting it would unfairly depress the result. Juke records every implementation it displaces with a proxy in a runtime registry (`JukeMockRegistry`), and the coverage analyzer skips exactly those classes. The developer never names an implementation class; the exclusion is a byproduct of the proxying Juke already does.

## 9.4 UI Coverage

Front-end coverage is produced outside the JVM. The SPA is built instrumented (`vite-plugin-istanbul`), Playwright harvests `window.__coverage__` after each test, and `nyc` renders a report. `GET /service/coverage/ui` reads that report's summary:

```
{ "available": true, "passed": true, "tool": "nyc/Istanbul", "lines": 78.0, "statements": 77.4,
  "functions": 80.0, "branches": 62.5, "reportUrl": "/coverage/ui/index.html" }
```

## 9.5 The Combined Endpoint

`GET /service/coverage` returns both halves in a single payload — useful for dashboards and one-call CI gates:

```
{ "server": { "available": true, "passed": true, "tool": "JaCoCo", ... }, "ui": { "available": true, "passed": true, "tool": "nyc/Istanbul", ... }, "passed": true, "generatedAt": "2026-05-20T10:00:00Z" }
```

Each half carries its own `available` and `passed` flags; the top-level `passed` is `true` only when both halves are. If only the server half is attached (no Playwright run yet) the UI half reports `available: false` and the top-level `passed` reflects only the half that gates.

## 9.6 Thresholds and the `passed` Flag

Every summary carries a `passed` boolean. By default it is `true` (no thresholds set). Configure minimums in `application.yml` to make it gate:

```
juke: coverage: threshold: server: line: 80 branch: 60 ui: lines: 70 branches: 55
```

When a metric falls short, `passed` flips to `false` and `message` names every metric that missed and by how much. A CI script can fail the build on a single `jq` call without parsing percentages:

```
curl -sf http://localhost:8080/service/coverage | jq -e '.passed'
```

`jq -e` exits non-zero when `passed` is `false`, so the call drops straight into a pipeline step.

## 9.7 Opt-In, Production-Safe

The whole feature is gated by `juke.coverage.enabled` (default `false`) and, for server coverage, the presence of the `-javaagent`. A production deployment sets neither and pays nothing — the coverage beans are never instantiated.

---

# Chapter 10. Session Reports

---

A single replay session is interesting; a suite of them is a *result*. The report endpoint consolidates every completed session for a track into one document.

## 10.1 The Report Endpoint

`GET /service/recording/report?track={name}` returns pretty-printed JSON:

```
{ "track": "demo", "label": "Greetings smoke test", "recordedAt": "2026-05-19T...", "sessions": [ {  
  "sessionId": "...", "description": "Normal replay", "callCount": 2, "overallStatus": "COMPLETED",  
  "calls": [ { "sequence": 1, "method": "greeting", "recordedArguments": ["Alice"],  
    "actualArguments": ["Alice"], "inputMatched": true } ] } ] }
```

## 10.2 Labels and Descriptions

Two optional query parameters feed the report:

- `record/start?track=...&label=...` — the track label, stored in `juke-metadata.json`.
- `session/start?track=...&description=...` — the per-session description.

Neither is required; both make a report readable months later.

## 10.3 Reading the Status

Each completed session carries an `overallStatus`:

| Status                    | Meaning                                                |
|---------------------------|--------------------------------------------------------|
| COMPLETED                 | Every call's actual arguments matched the recording.   |
| COMPLETED_WITH_DEVIATIONS | At least one call's input differed from the recording. |

The companion endpoint `GET /service/recording/inputs?track={name}` returns just the recorded input sidecars, for comparing what a run *should* have sent.

---

## Part II — Reference

---

---

# Chapter 11. The `/service/*` HTTP Contract

`juke-remix-rest-service` exposes the control surface below. All endpoints accept `GET` unless noted; responses are `application/json` unless an attachment is streamed.

## 11.1 Recording Control

| Endpoint                                        | Description                                                                                         |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>/service/record/start?track={name}</code> | Begin recording into the named track. Optional <code>&amp;label={text}</code> sets the track label. |
| <code>/service/record/end</code>                | Flush and close the ZIP; stream the bytes back as an attachment.                                    |

## 11.2 Replay Control

| Endpoint                                        | Description                                     |
|-------------------------------------------------|-------------------------------------------------|
| <code>/service/replay/start?track={name}</code> | Switch to replay, pinned at the named track.    |
| <code>/service/replay/disable</code>            | Pass through without leaving replay mode.       |
| <code>/service/replay/enable</code>             | Re-enable replay after a <code>disable</code> . |

## 11.3 Recordings Catalog

| Endpoint                                | Description                                               |
|-----------------------------------------|-----------------------------------------------------------|
| <code>/service/recordings</code>        | JSON array of every recording name the agent holds.       |
| <code>/service/recordings/{name}</code> | Stream the named recording's ZIP bytes; 404 when unknown. |

## 11.4 Sessions

| Endpoint | Description |
|----------|-------------|
|----------|-------------|



|                                                  |                                                                                                                        |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <code>/service/session/start?track={name}</code> | Start a per-session replay. Optional <code>&amp;description={text}</code> . Sets the <code>JUKE_SESSION</code> cookie. |
| <code>/service/session/stop</code>               | End the active session; clear the cookie.                                                                              |
| <code>/service/session/status</code>             | The active session's id, track, age, and step counter.                                                                 |
| <code>/service/sessions</code>                   | Read-only view of every active session ( <code>lastCall</code> , <code>percentComplete</code> ).                       |

## 11.5 Reports

| Endpoint                                            | Description                                                        |
|-----------------------------------------------------|--------------------------------------------------------------------|
| <code>/service/recording/report?track={name}</code> | Consolidated JSON report of every completed session for the track. |
| <code>/service/recording/inputs?track={name}</code> | The recorded input-argument sidecars for the track.                |

## 11.6 Coverage

Available only when `juke.coverage.enabled=true` (Chapter 9).

| Endpoint                              | Description                                                                                                      |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <code>/service/coverage</code>        | Combined summary — server and UI in one response; carries a top-level <code>passed</code> for one-call CI gates. |
| <code>/service/coverage/server</code> | Live server (JaCoCo) coverage summary; HTML at <code>/coverage/server/</code> .                                  |
| <code>/service/coverage/ui</code>     | Front-end (nyc/Istanbul) coverage summary; HTML at <code>/coverage/ui/</code> .                                  |

## 11.7 Remix

| Endpoint | Description |
|----------|-------------|
|----------|-------------|

|                                                                                                                               |                                                                                                                |
|-------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>/service/remix/exceptionSchedule?classAndMethodSequence={id}&amp;exceptionType={type}&amp;exceptionMessage={msg}</code> | Schedule one-shot exception (by the named recording entry).                                                    |
| <code>/service/remix/delaySchedule?classAndMethodSequence={id}&amp;delayBefore={ms}</code>                                    | Schedule a delay before the named entry returns.                                                               |
| <code>/service/remix/clear</code>                                                                                             | Clear all scheduled delays and exceptions, so a fault injected in one replay run does not carry into the next. |

## 11.8 Plugin Registry

| Endpoint                                      | Method | Description                              |
|-----------------------------------------------|--------|------------------------------------------|
| <code>/service/plugins/register</code>        | POST   | Register a plugin.                       |
| <code>/service/plugins/{id}/heartbeat</code>  | POST   | Liveness ping; rotates the plugin token. |
| <code>/service/plugins/{id}/deregister</code> | POST   | Voluntary teardown.                      |
| <code>/service/plugins</code>                 | GET    | List registered plugins.                 |
| <code>/service/plugins/capabilities</code>    | GET    | Union of advertised capabilities.        |
| <code>/service/plugins/{id}</code>            | GET    | Detail for one plugin.                   |

# Chapter 12. Configuration Reference

Juke reads layered YAML — `juke-defaults.yml` (in the jar) ← `application.yml` ← `application-{profile}.yml`. Every key has a `-Djuke.*` system-property alias; YAML is preferred for everything but one-off CI overrides.

## 12.1 Core Keys

| Key                                   | Default                              | Notes                                                                  |
|---------------------------------------|--------------------------------------|------------------------------------------------------------------------|
| <code>juke.enabled</code>             | <code>false</code>                   | Master toggle. Nothing activates unless <code>true</code> .            |
| <code>juke.path</code>                | <code>\${java.io.tmpdir}/juke</code> | Directory of recording ZIPs.                                           |
| <code>juke.mode</code>                | <code>ignore</code>                  | Global mode; usually flipped via REST.                                 |
| <code>juke.zip</code>                 | <i>(none)</i>                        | Default ZIP name for global record/replay.                             |
| <code>juke.tests</code>               | <i>(none)</i>                        | Whitelist of allowed track names.                                      |
| <code>juke.disabled</code>            | <code>false</code>                   | Soft kill-switch ( <code>juke.enabled=false</code> is preferred).      |
| <code>juke.args-validation</code>     | <code>warn</code>                    | <code>off</code> / <code>warn</code> / <code>strict</code> (see §5.4). |
| <code>juke.storage.folder.path</code> | <code>recordings</code>              | Folder root for the <code>JukeStorage</code> SPI.                      |
| <code>juke.storage.backend</code>     | <code>folder</code>                  | <code>db</code> activates the Enterprise JPA backend.                  |

## 12.2 Coverage Keys

Read by the optional `juke-coverage` module (Chapter 9).

| Key | Default | Notes |
|-----|---------|-------|
|-----|---------|-------|

|                                                         |                                               |                                                                   |
|---------------------------------------------------------|-----------------------------------------------|-------------------------------------------------------------------|
| <code>juke.coverage.enabled</code>                      | <code>false</code>                            | Opt-in toggle for the <code>/service/coverage/*</code> endpoints. |
| <code>juke.coverage.classes</code>                      | <code>(unset)</code>                          | Path to the application's <code>target/classes</code> .           |
| <code>juke.coverage.sources</code>                      | <code>(unset)</code>                          | Path to <code>src/main/java</code> for HTML source highlighting.  |
| <code>juke.coverage.report-dir</code>                   | <code>\${user.home}/juke-demo/coverage</code> | Where the server HTML report is written.                          |
| <code>juke.coverage.ui-report-dir</code>                | <code>\${user.home}/juke-demo/coverage</code> | Where the nyc UI report is read from.                             |
| <code>juke.coverage.bundle-name</code>                  | <code>Application under test</code>           | Display name for the server coverage bundle.                      |
| <code>juke.coverage.threshold.server.line</code>        |                                               | Minimum server line coverage (%). <code>0</code> = no gate.       |
| <code>juke.coverage.threshold.server.branch</code>      |                                               | Minimum server branch coverage (%).                               |
| <code>juke.coverage.threshold.server.instruction</code> |                                               | Minimum server instruction coverage (%).                          |
| <code>juke.coverage.threshold.ui.line</code>            | <code>0</code>                                | Minimum UI line coverage (%).                                     |
| <code>juke.coverage.threshold.ui.statements</code>      |                                               | Minimum UI statement coverage (%).                                |
| <code>juke.coverage.threshold.ui.functions</code>       |                                               | Minimum UI function coverage (%).                                 |
| <code>juke.coverage.threshold.ui.branches</code>        |                                               | Minimum UI branch coverage (%).                                   |

When any threshold is set, every coverage response carries `passed: false` the moment a metric falls short — see §9.6 for the CI-gate pattern.

## 12.3 The Per-Environment Pattern

```
# application.yml – base; no enabled key, so Juke is off juke: path: /tmp/juke-tracks #  
application-dev.yml juke: enabled: true # application-prod.yml – omit juke.enabled entirely
```

Activate with **SPRING\_PROFILES\_ACTIVE=dev**. Juke turns on or stays invisible with no code change.

---

# Chapter 13. The Storage SPI

---

`JukeStorage` is the framework's storage abstraction. A host can replace it without touching any other Juke code.

## 13.1 Two Abstraction Levels

- **Per-recording** — read and write entries within one ZIP (`readFromFile`, `writeToFile`, `getFileNames`, `path`).
- **Recording-store** — enumerate, load, and save whole recordings by name (`listRecordings`, `loadRecording`, `saveRecording`), used by the `/service/recordings` catalog.

## 13.2 The Default Implementation

`JukeZipDAOImpl` treats `juke.storage.folder.path` as the store and each `*.zip` file as a per-recording handle. It is registered by `JukeStorageAutoConfiguration` as `@Bean @ConditionalOnMissingBean`.

## 13.3 Custom Backends

Expose your own `JukeStorage` bean and the autoconfiguration steps aside. The Enterprise `juke-scenario-service` does exactly this: its `JukeJpaStorageBackend` activates with `juke.storage.backend=db` to keep recordings in a database. The agent code is identical either way.

---

## Part III — Appendices

---

---

# Appendix A. Sample Applications

---

The `juke-samples/` directory contains runnable references:

| Sample                               | Demonstrates                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>juke-sample-greeting</code>    | The canonical app — one REST endpoint, one <code>@Juke</code> seam, a bundled React (Vite) SPA. The basis of <i>DEMO.md</i> .                                                                                                                                                                                                                                               |
| <code>juke-sample-coverage</code>    | The functional-coverage demo (Chapter 9). A live dashboard polls <code>/service/coverage</code> while you click through the journey; bundled <code>demo-start-server.{ps1,bat}</code> and <code>demo-run-playwright.{ps1,bat}</code> launchers wire up the JaCoCo agent and the nyc report path.                                                                            |
| <code>juke-sample-todo</code>        | A plain REST CRUD surface; the false-positive / <code>@JukeIgnorable</code> walk-through.                                                                                                                                                                                                                                                                                   |
| <code>juke-sample-session</code>     | Per-session replay and the Playwright specs, including the visual demo.                                                                                                                                                                                                                                                                                                     |
| <code>juke-sample-annotations</code> | <code>@Juke</code> on fields, methods, and constructor parameters.                                                                                                                                                                                                                                                                                                          |
| <code>juke-sample-exceptions</code>  | Exception and latency flows (Chapter 8). A SKU order places three orders through a <code>@Juke</code> OMS seam, driven four times — record, deterministic replay, replay with an injected delay ("queued"), and replay with an injected exception ("technical difficulties"). Confirmation numbers are <code>@JukeIgnorable</code> ; coverage is shown in a separate popup. |

`juke-samples/DEMO.md` is a 15-minute guided walk-through built on `juke-sample-greeting`.

---



# Appendix B. Troubleshooting

---

**Recording is empty or only contains `juke.json`.** Confirm `juke.enabled: true`, that the global mode is `record`, that the `@Juke` bean is injected through Spring (not `new-ed`), and that the component scan covers `org.juke.framework`.

**`JukeReplayNotFoundException`.** The replay asked for a sequence the recording does not have — usually call-order drift between record and replay, or a `juke.zip` pointing at the wrong track. Re-record or fix the track name.

**INPUT MISMATCH warnings.** `juke.args-validation` found the live arguments differ from the recorded ones. Re-record, set the value to `off`, or set it to `strict` to fail loudly in CI.

**H2 / database errors at startup.** A Community build is autoconfiguring JPA — `juke-scenario-service` or `spring-boot-starter-data-jpa` is on the classpath unintentionally. Check `mvn dependency:tree`.

**Tracks bleed across Playwright workers.** You are using `replay/start` for a process-wide flip instead of the per-session API. Switch to `/service/session/start` so each worker carries its own `JUKE_SESSION` cookie.

**`/service/coverage/server` reports "agent not attached".** The server was started without `-javaagent:jacoco-agent.jar`. Coverage degrades gracefully — attach the agent and retry.

---

# Appendix C. Glossary

---

## Track

A named recording — one ZIP archive of captured interactions.

## Seam

The `@Juke`-annotated injection point where interception happens.

## Mode

`record`, `replay`, `ignore`, `disable`, or `juke` — what the proxy does at the seam.

## Session

A cookie-scoped, per-track replay context, isolated from other sessions on the same JVM.

## Sequence

The 1-based per-`(type, method)` counter that orders recorded calls.

## Sidecar

An `*.args.json` or `*.type.json` file accompanying a response in the ZIP.

## Remix

Runtime fault injection — scheduled exceptions and delays on named entries.

## Functional coverage

Coverage measured while real user-journey tests drive a Juke-replaying server, as distinct from unit-test coverage.

---

# Colophon

---

The animal on the cover of *Juke in a Nutshell* is a **superb lyrebird** (*Menura novaehollandiae*), a ground-dwelling songbird of southeastern Australia. The lyrebird is among the finest vocal mimics in nature: it listens to the sounds of its environment — other birds, and sometimes mechanical noise — and reproduces them later with uncanny fidelity. A creature whose whole craft is *recording the real world and replaying it on demand* is a fitting mascot for Juke.

The cover image is a nineteenth-century engraving. The text is set in a serif body face with `Constant Width` for code.

*Juke in a Nutshell* — Early Release. Covers Juke 0.0.1. © The Juke Project, licensed under Apache 2.0.